

HashTable in Second Approach (Хэширование во втором подходе)

```
----- HashTable.h -----
#pragma once

#include "Person.h"

struct Cell {
    Person* ptr = nullptr;
    bool state = false;

    Cell() = default;
    Cell(Person* p, bool st) : ptr(p), state(st) {}
};

class HashTable {
    std::vector<Cell> table;
    size_t max_size;
    size_t size;

public:
    HashTable(size_t max_size) : max_size(max_size), size(0) {
        table.resize(max_size);
    }
    size_t hash(const std::string& key) const {
        size_t res = 0;
        for (char c : key)
            res += c;
        return res % max_size;
    }
    void reHash();
    bool add(Person* p);
    bool remove(const std::string& key);
    bool find(const std::string& key, Person*& out);
    void fill(std::ifstream& file);
    void print() const;
};
```

```
----- HashTable.cpp -----
#include "HashTable.h"
#include <iostream>
#include <sstream>
#include "Person.h"

void HashTable::reHash()
{
    size_t new_max_size = max_size * 2;
    std::vector<Cell> new_table(new_max_size);

    for (const auto& cell : table) {
        if (cell.state) {
            size_t index = hash(cell.ptr->getName()) % new_max_size;
            while (new_table[index].state) {
                index = (index + 1) % new_max_size;
            }
            new_table[index] = cell;
        }
    }
}
```

```

    }
}
table = std::move(new_table);
max_size = new_max_size;
}

bool HashTable::add(Person* p) {
    if ((double)size / max_size >= 0.7)
        reHash();

    size_t index = hash(p->getName());
    bool res = false;

    // Linear probing to find an empty slot or matching key
    while (table[index].state && table[index].ptr->getName() != p->getName()) {
        index = (index + 1) % max_size;
    }

    if (!table[index].state) {
        table[index].ptr = p;
        table[index].state = true;
        size++;
        res = true;
    }

    return res;
}

bool HashTable::remove(const std::string& key) {
    size_t index = hash(key);
    int window = -1;

    auto dist = [this](int a, int b) {
        return (b - a + max_size) % max_size;
    };

    while (table[index].state && table[index].ptr->getName() != key)
        index = (index + 1) % max_size;

    delete table[index].ptr;
    table[index].ptr = nullptr;
    table[index].state = false;

    window = index;
    index = (index + 1) % max_size;

    while (table[index].state) {
        if (dist(hash(table[index].ptr->getName()), index) >= dist(window, index)) {
            table[window] = table[index];
            window = index;
            table[index].state = false;
            table[index].ptr = nullptr;
        }
        else {
            index = (index + 1) % max_size;
        }
    }
}

```

```

        --size;
        return true;
    }

    bool HashTable::find(const std::string& key, Person*& out) {
        size_t index = hash(key);
        while (table[index].state && table[index].ptr->getName() != key)
            index = (index + 1) % max_size;

        if (table[index].state) {
            out = table[index].ptr;
            return true;
        }

        return false;
    }

    void HashTable::fill(std::ifstream& file) {
        std::string line;

        while (std::getline(file, line)) {
            if (line == "Patient") {
                std::string name, gender, policy;
                int age;
                std::getline(file, name);
                std::getline(file, gender);
                file >> age;
                file.ignore();
                std::getline(file, policy);
                std::getline(file, line);

                Person* p = new Patient({ name, gender, age, policy });
                add(p);
            }
            else if (line == "MedWorker") {
                std::string name, gender, specialization, degree;
                int exper;
                std::getline(file, name);
                std::getline(file, gender);
                std::getline(file, specialization);
                file >> exper;
                file.ignore();
                std::getline(file, degree);
                std::getline(file, line);
                Person* p = new Med_worker({ name, gender, specialization, exper, degree
});
                add(p);
            }
        }
    }

    void HashTable::print() const {
        for (size_t i = 0; i < max_size; ++i) {
            if (table[i].state) {
                size_t key = hash(table[i].ptr->getName()); // calculate hash key
                std::cout << i << " (hash: " << key << "): ";
                table[i].ptr->print();
            }
        }
    }

```

```

    }
    std::cout << "\n";
}
}

```

----- Person.h -----

```

#pragma once
#include <iostream>
#include <string>
#include <fstream>
#include <vector>

class Person {
protected:
    std::string name;
    std::string gender;

public:
    Person(std::string _name, std::string _gender) : name(_name), gender(_gender) {}

    std::string getName() const { return name; }
    std::string getGender() const { return gender; }

    virtual void print(std::ostream& out = std::cout) = 0;
};

class Patient : public Person {
private:
    int age;
    std::string med_Policy_Num;

public:
    Patient(std::string _name, std::string _gender, int _age, std::string policy)
        : Person(_name, _gender), age(_age), med_Policy_Num(policy) {}

    void print(std::ostream& out = std::cout);
};

class Med_worker : public Person {
private:
    std::string qualification;
    int experience;
    std::string category;

public:
    Med_worker(std::string _name, std::string _gender, std::string qual, int exp,
std::string cat)
        : Person(_name, _gender), qualification(qual), experience(exp),
category(cat) {}

    void print(std::ostream& out = std::cout);
};

```

----- Person.cpp -----

```
#include "Person.h"

void Patient::print(std::ostream& out)
{
    out << "Information about the patient: \n"
        << "Name: " << name << '\n'
        << "Gender: " << gender << '\n'
        << "Age: " << age << '\n'
        << "medical Policy Number : " << med_Policy_Num << '\n';
}

void Med_worker::print(std::ostream& out)
{
    out << "Information about the Medworker: \n"
        << "Name: " << name << '\n'
        << "Gender: " << gender << '\n'
        << "Qualification: " << qualification << '\n'
        << "experience: " << experience << '\n'
        << "category: " << category << '\n';
}
```

----- Source.cpp -----

```
#include "Person.h"
#include "HashTable.h"
#include <fstream>

int main()
{
    std::ifstream file("file.txt");
    if (file)
    {
        HashTable HT(3);
        HT.fill(file);
        HT.print();

        /*std::cout << "----- Finding the Person ----- \n";
        Person* pt = nullptr;
        if (HT.find("Петрова_Анна", pt))
            pt->print();
        else
            std::cout << "Not found \n";
        std::cout << "----- \n";*/

        std::cout << "----- Adding the Person ----- \n";
        if (!HT.add(new Med_worker({ "Петрова_Анна", "мужчина", "хирург",
11, "Second"})))
            std::cout << "No add \n";
        HT.print();
        std::cout << "----- \n";

        /*std::cout << "----- Remving the Person ----- \n";
        HT.remove("Сидоров_Сергей133");
        HT.print();
        std::cout << "----- \n";*/
```

```

    }
    else
        std::cout << "File is not opened!\n";

    file.close();

    std::cin.ignore();
    return 0;
}

```

----- file.txt -----

Patient
Иванов_Иван
мужчина
30
123456789

MedWorker
Сидоров_Сергей
мужчина
хирург
10
first

Patient
Петрова_Анна
женщина
25
987654321

MedWorker
Ковалева_Мария
женщина
хирург
15
higher

MedWorker
Сидоров_Сергей1
мужчина
хирург
10
higher

MedWorker
Сидоров_Сергей11
мужчина
хирург
10
second

MedWorker
olga
женщина
хирург
10
higher

MedWorker
Сидоров_Сергей133
мужчина
хирург
10
second

MedWorker
Маруаат
женщина
хирург
10
second
